Trade-off between Small and Large Transformers for German NER Fine-Tuning

Luca Sander

January 6, 2026

Abstract

TODO

1 Introduction

TODO

2 Problem Statement and Research Questions

TODO

3 Background

TODO

4 Related Work

TODO

5 Dataset

The dataset we use in this study is the GermEval 2014 NER dataset [3], which builds upon the annotation published by Benikova [2], which after some adjustments became the dataset used for the GermEval 2014 shared task [1] and was downloaded from the Hugging Face datasets library. The dataset contains German text from Wikipedia and news articles, annotated with named entities in a nested format. In total the dataset contains over 30,000 sentences with approximately 590,000 tokens. The dataset is split into:

- Training set: 24,002 sentences
- Development set: 2,200 sentences
- Test set: 5,100 sentences

The dataset includes annotations for 4 main entity types: *Person*, *Organization*, *Location*, and *Other*. Each entity type also has subtypes, namely *part* and *deriv*, which leads to a total of 12 distict entity labels. The dataset also uses nested annotations, meaning that entities can be contained within other entities. An example of that would be *"FC BAYERN MUNICH"*, where *"MUNICH"* is firstly annotated as an *Organization*, but in a nested annotations it is also annotated as a *Location*. The distribution of the annotated entities in the training set is shown in Table 1.

Entity Type	**All**	**Nested**
Location	12,204	1,454
Location deriv	4,412	808
Location part	713	39
Person	10,517	488
Person deriv	95	20
Person part	275	29
Organization	7,182	281
Organization deriv	56	4
Organization part	1,077	9
Other	4,047	57
Other deriv	294	3
Other part	252	2
Total	**41,124**	**3,194**

Table 1: Distribution of entity types in the GermEval 2014 dataset (outer and nested annotations).

However, for the purpose of this study, we only focus on the outermost entities and ignore the nested annotations. Concretely, only the primary annotation layer (outer entities) is retained, while the nested entity layer is not used. This choice reduces the task to a standard flat named entity

recognition setting, where each token is assigned at most one entity label. This follows the more general application of transformer-based Named Entity Recognition, where other datasets often do not have the nested elements, such as the CoNLL-2003 dataset [9] used for example in English NER. [7].

6 Approach

6.1 Models

For this study, we selected three transformer-based models that vary in size and purpose. All models were downloaded from the Hugging Face model hub. The models chosen are:

- **bert-base-german-cased** [4]: The German BERT base model is a transformer model published in 2019, pretrained on roughly 12GB of German text data, sensitive to cased input, and evaluated on various tasks including CONLL03, GermEval14, and GNAD. The model has 12 layers, each with 12 attention heads, the tokens are embedded into a 768-dimensional space, with the feed-forward layers having a dimensionality of 3072. The vocabulary size of the model is 30,000 tokens, leading to a total of around 110 million parameters.

- **distilbert-base-german-cased** [6]: DistilBERT is a compressed version of BERT trained through knowledge distillation, where a smaller student model is trained to match the behavior of a larger teacher model. The DistilBERT model has the same hidden dimensionality, feed-forward size, and number of attention heads as the BERT base model, but reduces the number of transformer layers from 12 to 6, resulting in a significantly smaller and faster model, while retaining the same width. The model uses a vocabulary of 31,102 tokens and contains approximately 67 million parameters.

- **bert-base-multilingual-cased** [5]: The multilingual BERT base model is a transformer model released by Google, pretrained with a masked language modeling objective on the Wikipedias of 104 languages (including German), and is case sensitive. It follows the BERT-base architecture with 12 layers and 12 attention heads, a hidden size of 768 and a feed-forward size of 3072. Due to its much larger vocabulary (119,547 tokens), the model has approximately 179 million parameters.

6.2 Training Strategies

We use two different training strategies for fine-tuning all models on the GermEval14 dataset. In all cases, we add a token classification head as a linear layer on top of the transformer to predict the entity labels for each token. We then experiment with the following strategies for training:

- **Full Fine-Tuning**: In this strategy, we train the entire model, including all transformer layers and the classification head, on the GermEval14 training set. This allows the model to adapt all its parameters to the specific NER task.

- **Frozen Encoder**: In this strategy, we freeze all the transformer layers and only train the classification head on top. This means that the pre-trained weights of the transformer remain unchanged, and only the final layer is updated during training. This approach is computationally more efficient and requires less memory, as only the parameters of the classification head need to be stored and updated.

We also experiment with an additional variant where we concatenate the last four hidden layers of the transformer as input to the classification head, instead of just using the final layer's output. This approach needs a modification of the classification head to accommodate the increased input dimensionality, so its size is $4 * 768 = 3072$.

The specific hyperparameter configurations for each strategy were selected based on a small grid search on the development set, as described in Section 8.

6.3 Implementation Details

Label set and preprocessing The label inventory is derived from the GermEval14 dataset by collecting all unique BIO tags (`ner_tag`) across all dataset splits. A deterministic mapping between labels and numeric IDs (`label2id` / `id2label`) is created and passed to the model configuration to ensure consistent decoding and evaluation.

Tokenization and label alignment All models use their respective tokenizers via `AutoTokenizer`. Since the dataset provides word-level token sequences, tokenization is performed with `is_split_into_words=True`, applying truncation and fixed-length padding up to a maximum sequence length of 128. Label alignment is implemented using the tokenizer's `word_ids()` mapping: special tokens and padding positions are masked with the ignore index `-100`, while each subword token inherits the BIO label of its originating word. This ensures that loss and evaluation are computed only over valid token positions.

Model setup Models are instantiated using `AutoModelForTokenClassification`. Depending on the training strategy, either all model parameters are updated (full fine-tuning) or only the parameters of the token classification head are optimized while the transformer encoder is frozen.

Training Loop Training is implemented using the Hugging Face Trainer API, which provides a standardized training and evaluation loop for transformer models. Evaluation is performed at the end of each epoch on the development set. Final model performance is assessed on the test set; the evaluation metrics and protocol are described in detail in Section 7.

7 Evaluation

7.1 Metrics (GermEval14)

Benikova et al.[1] define four metrics for the shared task, but describe three main evaluation variants for nested NER. We summarize the three proposed metrics below.

Let P_1 and G_1 be the predicted/gold sets of outer (first-level) named entity chunks, and P_2 and G_2 the predicted/gold sets of inner (second-level) chunks. [1]

- **Metric 1: Strict, Combined Evaluation (official)**. All markables across both levels are evaluated jointly, distinguishing all 12 labels (including `deriv` and `part` subtypes). A predicted chunk matches a gold chunk iff (i) label matches, (ii) span matches, and (iii) the level (outer vs. inner) matches: $P = P_1 \cup P_2, \quad G = G_1 \cup G_2, \quad p == g \Leftrightarrow \text{label}(p) = \text{label}(g) \wedge \text{span}(p) = \text{span}(g) \wedge \text{level}(p) = \text{level}(g)$.
- **Metric 2: Loose, Combined Evaluation**. Same as Metric 1, but subtypes are collapsed (base label match is sufficient, e.g. `PER` matches `PERderiv`), while span and level must still match: $P = P_1 \cup P_2, \quad G = G_1 \cup G_2, \quad p == g \Leftrightarrow \text{baseLabel}(p) = \text{baseLabel}(g) \wedge \text{span}(p) = \text{span}(g) \wedge \text{level}(p) = \text{level}(g)$.
- **Metric 3: Strict, Separate Evaluation**. Two separate strict evaluations are reported: one for outer entities (P_1 vs. G_1) and one for inner entities (P_2 vs. G_2), each using the traditional strict match definition (label + exact span).

7.2 What we evaluate in this study

In our experiments we report:

- **Entity-level Precision/Recall/F1** computed from predicted and gold chunks of the outer entities, using strict matching (label + exact span). This corresponds to Metric 3's outer evaluation only, focusing on the primary task of flat NER.
- **Training Runtime** (wall-clock) for each experiment, to quantify efficiency trade-offs between model size and training strategy.
- **Inference Runtime** (wall-clock) on the test set, to assess practical deployment efficiency.

7.3 Evaluation protocol and outer-only choice

The GermEval dataset provides two annotation layers and the shared task evaluates both levels. However, in this study we only evaluate the outer entities and ignore the nested layer, as this would add complexity that is not central to the research questions.

Concretely, we evaluate predicted chunks against G_1 (outer gold entities) and do not model nor score G_2 (inner entities). This choice is motivated by:

- **Task alignment and comparability**: most widely used NER benchmarks - including the

CoNLL-2003 dataset [9] - assume flat annotation; focusing on outer entities reduces GermEval to this standard setting.

- **Modeling scope**: nested prediction introduces an additional structural requirement (two-level outputs) beyond plain sequence tagging; our experiments target the trade-off between model size and fine-tuning strategy under a standard token-classification setup.
- **Data characteristics**: only a minority of entities are nested (about 7.8% in the dataset statistics reported by the task paper), while inner entities show substantially lower performance in the shared task analysis, suggesting higher difficulty. [1]

8 Experimental Setup

8.1 Hardware and environment

All experiments were run on a single workstation equipped with an NVIDIA RTX 3080 GPU. Even though better hardware is available, the experiments still give insight into the relative efficiency of the models and training strategies.

8.2 Hyperparameter selection via small grid search

To ensure a fair and computationally feasible comparison across all six model configurations (three architectures $\times$ frozen vs. full fine-tuning), we performed a small but targeted hyperparameter search on a single reference model: bert-base-german-cased. We conducted the search separately for the two training regimes:
(i) **Full fine-tuning** (encoder + classifier head trainable), and
(ii) **Frozen encoder** (only the classifier head trainable).
We did not perform hyperparameter tuning for the classification using a concatenation of the last 4 layers, as this would remove comparability between the configurations.

To keep exploratory runs efficient, we trained on a reduced subset of 50% of the training split while always evaluating on the full development set. For each candidate configuration, we trained for the specified number of epochs and selected the setting with the highest development F1-score (outer entities, strict matching as described in Section 7). We additionally recorded wall-clock runtime to ensure the chosen settings were not only accurate but also practically feasible.

Search space. For full fine-tuning, we tested learning rates $\{2 \times 10^{-5}, 3 \times 10^{-5}, 5 \times 10^{-5}\}$ and epochs $\{3, 4, 5\}$. For the frozen encoder regime, we tested higher learning rates - $\{3 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}, 2 \times 10^{-3}\}$ - appropriate for training only a shallow head, and epochs in a slightly wider range - $\{5, 8, 10\}$ - to allow convergence under frozen representations.

Selected hyperparameters. We selected the following hyperparameters based on the grid search results:

- **Full fine-tuning:** learning rate 3×10^{-5}, 4 epochs.
- **Frozen encoder:** learning rate 2×10^{-3}, 8 epochs.

The full grid-search results are reported in Appendix A (Table 2). Even though the best results were achieved with 10 epochs in the frozen regime, we selected 8 epochs to limit runtime while still achieving near-optimal performance as the difference is only marginal.

These hyperparameters were then used unchanged for all models within the corresponding training regime to preserve comparability between architectures and to avoid confounding results with per-model tuning.

9 Results

TODO

10 Discussion

TODO

11 Error Analysis

TODO

12 Limitations

TODO

13 Conclusion and Future Work

TODO

References

[1] Darina Benikova, Chris Biemann, Max Kisselew, and Sebastian Pado. GermEval 2014 Named Entity Recognition Shared Task: Companion Paper. .

[2] Darina Benikova, Chris Biemann, and Marc Reznicek. NoSta-D Named Entity Annotation for German: Guidelines and Dataset. .

[3] Darina Benikova et al. Germeval 2014 named entity recognition dataset. https://huggingface.co/datasets/lschoen/germeval14_ner, 2014. Hugging Face dataset repository, accessed 2025.

[4] Google Research. bert-base-german-cased. https://huggingface.co/google-bert/bert-base-german-cased, 2019. Pre-trained BERT base model for German (cased), accessed 2025.

[5] Google Research. bert-base-multilingual-cased. https://huggingface.co/google-bert/bert-base-multilingual-cased, 2019. Pre-trained multilingual BERT model (cased), accessed 2025.

[6] Hugging Face. distilbert-base-german-cased. https://huggingface.co/distilbert/distilbert-base-german-cased, 2020. Pre-trained DistilBERT model for German (cased), accessed 2025.

[7] Navjeet Kaur, Ashish Saha, Makul Swami, Muskan Singh, and Ravi Dalal. BertNer: A Transformer-Based Approach For Named Entity Recognition. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–7, June 2024. doi: 10.1109/ICCCNT61001.2024.10724703. URL https://ieeexplore.ieee.org/document/10724703/. ISSN: 2473-7674.

[8] Raphael Scheible, Johann Frei, Fabian Thomczyk, Henry He, Patric Tippmann, Jochen Knaus, Victor Jaravine, Frank Kramer, and Martin Boeker. GottBERT: a pure German Language Model. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21237–21250, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.1183. URL https://aclanthology.org/2024.emnlp-main.1183/.

[9] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147, 2003. URL https://aclanthology.org/W03-0419/.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

A Hyperparameter Search Results

Regime	LR	Epochs	Val Prec.	Val Rec.	Val F1	Runtime (min)
			Full fine-tuning			
Full FT	2×10^{-5}	3	0.8093	0.8316	0.8203	4.9
Full FT	2×10^{-5}	4	0.8146	0.8341	0.8243	6.4
Full FT	2×10^{-5}	5	0.8173	0.8433	0.8301	8.2
Full FT	3×10^{-5}	3	0.8191	0.8453	0.8320	4.9
Full FT	3×10^{-5}	4	0.8244	0.8456	0.8349	6.5
Full FT	3×10^{-5}	5	0.8164	0.8385	0.8273	8.1
Full FT	5×10^{-5}	3	0.8217	0.8396	0.8306	4.8
Full FT	5×10^{-5}	4	0.8197	0.8312	0.8254	6.5
Full FT	5×10^{-5}	5	0.8142	0.8349	0.8244	8.2
			Frozen encoder			
Frozen	3×10^{-4}	5	0.6473	0.6081	0.6271	3.3
Frozen	3×10^{-4}	8	0.6578	0.6266	0.6418	5.5
Frozen	3×10^{-4}	10	0.6572	0.6330	0.6449	6.8
Frozen	5×10^{-4}	5	0.6528	0.6251	0.6387	3.5
Frozen	5×10^{-4}	8	0.6675	0.6429	0.6549	5.6
Frozen	5×10^{-4}	10	0.6665	0.6472	0.6567	6.7
Frozen	1×10^{-3}	5	0.6657	0.6489	0.6572	3.1
Frozen	1×10^{-3}	8	0.6746	0.6551	0.6647	5.0
Frozen	1×10^{-3}	10	0.6725	0.6577	0.6650	6.2
Frozen	2×10^{-3}	5	0.6671	0.6567	0.6619	3.3
Frozen	2×10^{-3}	8	**0.6758**	**0.6598**	**0.6677**	5.1
Frozen	2×10^{-3}	10	0.6743	0.6630	0.6686	6.3

Table 2: Grid-search results for the `bert-base-german-cased` reference model. Each row reports validation precision/recall/F1 (outer entities, strict matching) and runtime for one learning rate (LR) and epoch setting.